Hw 4

Jack Vranicar

November 3, 2025

1 Q1

Table of Contents

.... 1
Complex Step 1
Analytical 1
Plotting 2
Functions 3


```
clc;  clearvars; close all;

% z= K / (1 + (K/z0 - 1) exp(rt))

% z' = IM[z(x + ih)] / h
t_vals = 0:.01:3;

r = sin(t_vals);
K = exp(t_vals);
z0 = 3 * ones(1, numel(t_vals));
theta = [r; K; z0];

h = 1e-12;
```

Complex Step

```
z_p_r = complexStep(theta, 1, h, t_vals);
z_p_K = complexStep(theta, 2, h, t_vals);
z_p_z0 = complexStep(theta, 3, h, t_vals);

z_ps = [z_p_r, z_p_K, z_p_z0];
names = ['R', 'K', "Z0"];
```

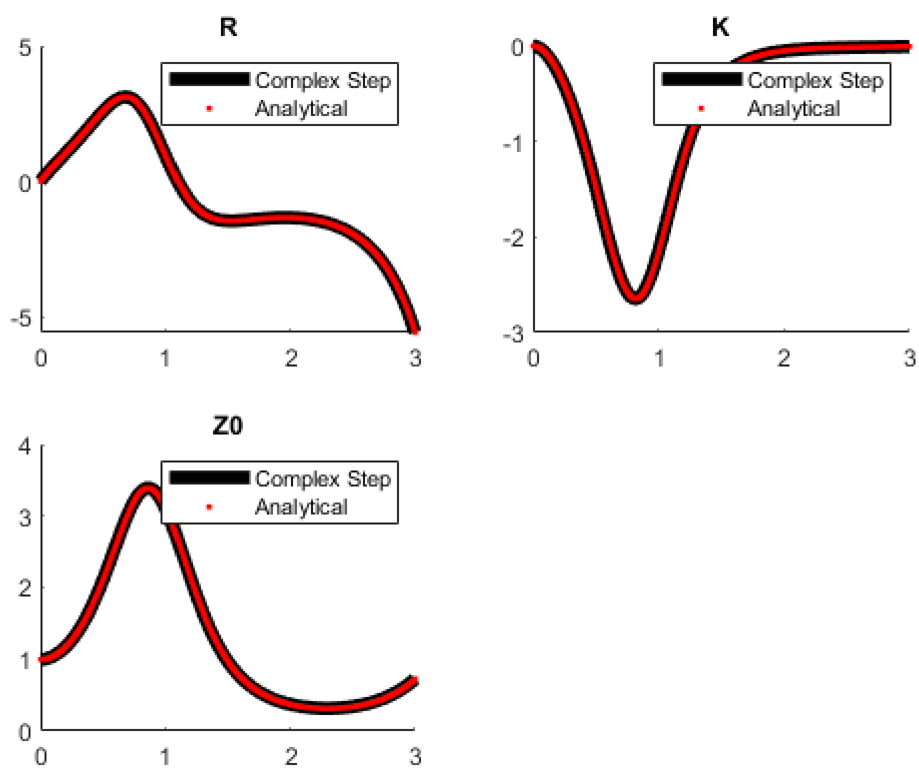
Analytical

```
syms r K z0 t

z = K / (1 + (K/z0 - 1) * exp(r * t));

% dz/dr
dz_dr = simplify(diff(z, r));

%dz/dK
dz_dK = simplify(diff(z, K));

% dZ/dz0
dz_dZ0 = simplify(diff(z, z0));

z_ = zeros(3, numel(t_vals));

for i = 1:numel(t_vals)
```

```
    z_(1, i) = subs(dz_dr, [r, K, z0, t], [theta(1, i), theta(2, i), 
theta(3, i), t_vals(i)]);
    z_(2, i) = subs(dz_dK, [r, K, z0, t], [theta(1, i), theta(2, i), 
theta(3, i), t_vals(i)]);
    z_(3, i) = subs(dz_dZ0, [r, K, z0, t], [theta(1, i), theta(2, i), 
theta(3, i), t_vals(i)]);
end
```

Plotting

```
figure()
for i = 1:3
    subplot(2, 2, i)
    hold on
    plot(t_vals, z_ps(:, i), 'k',  'DisplayName', 'Complex Step', 
'LineWidth', 5);
    plot(t_vals, z_(i,:), 'r.', 'DisplayName', 'Analytical', 'LineWidth', 1);
    hold off
    legend()
    title(names(i));
end
```



Functions

```
function z_p = complexStep(thetas, pos, h, t_vals)
    z_p = zeros(numel(t_vals), 1);
    thetas(pos, :) = thetas(pos, :) + 1i * h;

    for i = 1:numel(t_vals)
        t_val = t_vals(i);
        z = xfr(thetas(:, i), t_val);
        z_p(i) = imag(z) / h;
    end

end

function z = xfr(theta, t)
    r = theta(1);
    K = theta(2);
    z0 = theta(3);

    z = K / (1 + (K/z0 - 1) * exp(r * t));

end
```

Published with MATLAB® R2024b

2 Q2

Table of Contents

.... 1
Pt 1 1
Pt 2 1
Part 3 2
Part 4 2


```
clc; clear all; close all

% pg 186

% theta = [r; K; z0]
```

Pt 1

```
thresh = 1e-6; % nu
```

Pt 2

i is the numel of time j is numel of params

```
time = 1:10;
i = numel(time);
j = 3;

syms r K z0 t

theta = [r; K; z0];

z = K / (1 + (K/z0 - 1) * exp(r * t));

%dz/dr
dz_dr = simplify(diff(z, r));

%dz/dK
dz_dK = simplify(diff(z, K));

% dZ/dz0
dz_dZ0 = simplify(diff(z, z0));

derivs = [dz_dr; dz_dK; dz_dZ0];

theta_names = {"r"; "K"; "z0"; "t"};

for ii = 1:i % time

    theta_now = [output_function(time(ii)); time(ii)];
```

```
    for jj = 1:j % theta
        theta_star_j = theta_now(jj);
        df_dth_j_eqn = derivs(jj, 1);
        df_dth_j = subs(df_dth_j_eqn, theta_names, theta_now);
        S(ii, jj)= double(theta_star_j * df_dth_j);

    end
end

function out = output_function(time)
    r = sin(time);
    K = exp(time);
    z0 = 3;
    out = [r; K; z0];
end
```

Part 3

```
F = transpose(S) * S;
[F_eig_vect, F_eig] = eig(F);

[F_eig, I] = sort(diag(F_eig), 'descend');

F_eig_sorted(:, 1) = F_eig_vect(:, I(1));
F_eig_sorted(:, 2) = F_eig_vect(:, I(2));
F_eig_sorted(:, 3) = F_eig_vect(:, I(3));
```

Part 4

```
lam_1 = F_eig(1);
lam_p = F_eig(end);

m = [];
L = [];

if (lam_p/lam_1 > thresh)

    disp("All vars are identifiable");
else
    for i = 1:numel(F_eig) - 1
        % find m
        if ((F_eig(i)/lam_1) > thresh) && ((F_eig(i + 1)/lam_1) <=thresh)
            m = i;
        end

        % eigenvector corresponding with p
        vp = F_eig_sorted(:, end);
        [~,j] = max(vp);
        L = [L;j];
    end

    % This is part b. The algorithm is unclear on what needs to be
    % repeated, but this is my interpretation
```

```
    % now, repeat for other indices

    for t = 1:(m+1)
    % eigenvector corresponding with cur index
        vp = F_eig_sorted(:, t);
        [~,j] = max(vp);
        L = [L;j];
    end

    L = unique(sort(L));

    disp("The following indices are unidentifiable: " + string(L))

end
```

```
All vars are identifiable
```

Published with MATLAB® R2024b

3 Q3

Table of Contents

.... 1
Complex Step 1
Analytical 1
Plotting 1
Functions 2


```
clc;  clear all; close all;

t_vals = 0:.01:3;

P = sin(t_vals);
c = exp(t_vals);

theta = [P; c];

h = 1e-12;
```

Complex Step

```
z_p_P = complexStep(theta, 1, h, t_vals);
z_p_c = complexStep(theta, 2, h, t_vals);

z_ps = [z_p_P, z_p_c];
names = ['P', 'c'];
```

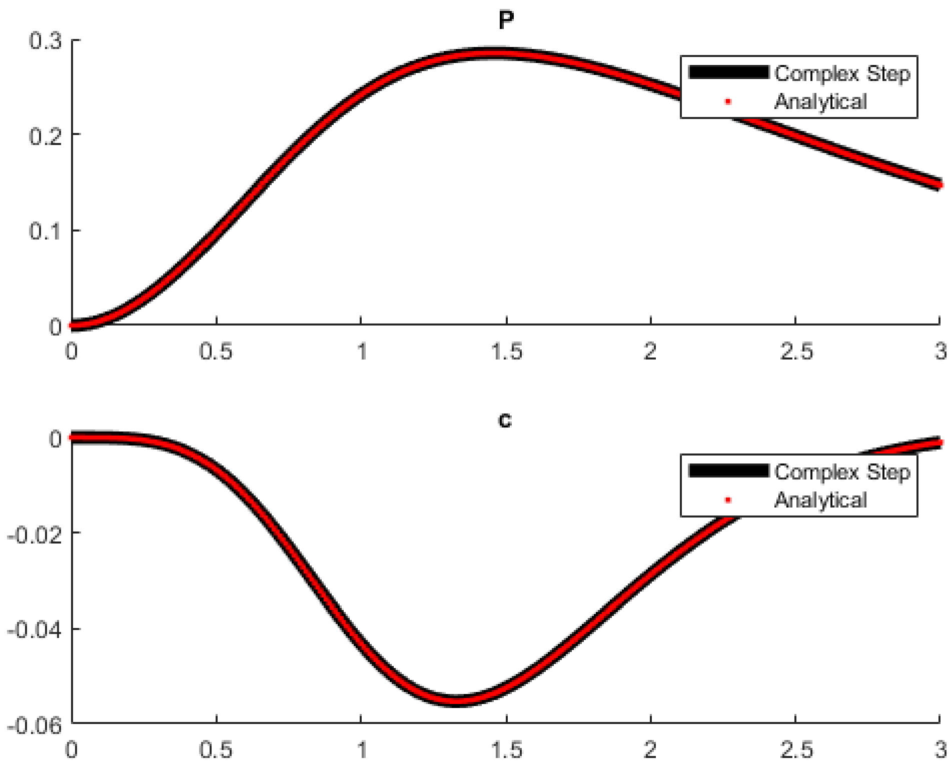
Analytical

```
syms P c t
t0 = 0;
z = (P/c^2) * (exp(-c*(t-t0)) - 1 + c*(t-t0));

% dz/dP
dz_dP = simplify(diff(z, P));

%dz/dc
dz_dc = simplify(diff(z, c));

z_ = zeros(2, numel(t_vals));

for i = 1:numel(t_vals)
    z_(1, i) = subs(dz_dP, [P, c, t], [theta(1, i), theta(2, i), t_vals(i)]);
    z_(2, i) = subs(dz_dc, [P, c, t], [theta(1, i), theta(2, i), t_vals(i)]);
end
```

Plotting

```
figure()
for i = 1:2
```

```
    subplot(2, 1, i)
    hold on
    plot(t_vals, z_ps(:, i), 'k',  'DisplayName', 'Complex Step',
'LineWidth', 5);
    plot(t_vals, z_(i,:), 'r.', 'DisplayName', 'Analytical', 'LineWidth', 1);
    hold off
    legend()
    title(names(i));
end
```



Functions

```
function z_p = complexStep(thetas, pos, h, t_vals)
    z_p = zeros(numel(t_vals), 1);
    thetas(pos, :) = thetas(pos, :) + 1i * h;

    for i = 1:numel(t_vals)
        t_val = t_vals(i);
        z = xfr(thetas(:, i), t_val);
        z_p(i) = imag(z) / h;
    end

end

function z = xfr(theta, t)
    t0 = 0;
```

```
    P = theta(1);
    c = theta(2);

    z = (P/c^2) * (exp(-c*(t-t0)) - 1 + c*(t-t0));

end
```

Published with MATLAB® R2024b

4 Q4

Table of Contents

.... 1
Partial Variance, Sobol Indices 1
Morris screening algorithm 9.21 1
Plotting 2
Fns 3


```
clc; clearvars; close all
```

Partial Variance, Sobol Indices

```
syms theta_sym [3 1]

f = myIshigami(theta_sym);

% for i = 1:3
%     f_ints(i) = int(f, theta_sym(i));
% end

%* Eqn 9.9

f0_ = int(int(int(f, theta_sym(1), 0, 1), theta_sym(2), 0, 1), theta_sym(3),
0, 1);
f0 = double(f0_);

range = 1:3;
for i = 1:3
    not_i = range(range~=i);
    f_i_th_i(i) = int(int(f, theta_sym(not_i(1)), 0, 1),
theta_sym(not_i(2)), 0, 1) - f0;
end

%* Eqn 9.17

for i = 1:3
    D_= int( (f_i_th_i(i))^2, theta_sym(i), 0, 1);
    D(i) = double(D_);
end

S = D/sum(D); % this equals 1 so should be good :)
```

Morris screening algorithm 9.21

```
test_points = 2:1000;
delta = 1e-4;
for a = 1:numel(test_points)
    %** 1
```

```
    R = test_points(a);

    %** 2
    d_j_i = [];
    p = 3;
    ei = zeros(p, 1);
    for j = 1:R
        theta_j = rand(p, 1); % random sample of theta

        for i = 1:numel(theta_j)

            ei(i) = 1;
            d_j(i) = (myIshigami(theta_j + delta*ei) - myIshigami(theta_j))/
delta;
            ei(i) = 0;
        end
        d_j_i = [d_j_i; d_j];
        d_j = [];

    end


    %** 3
    mu = sum(d_j_i) / R;

    sigmas = [0 0 0];

    for ii = 1:p % parameters

        for jj = 1:R % Samples
            tmp = (d_j_i(jj, ii) - mu(ii))^2;
            sigmas(ii) = sigmas(ii) + tmp;
        end
        sigmas(ii) = sqrt(sigmas(ii) * (1/(R-1)));
    end

    mus(a, :) = mu;
    sigmas_total(a, :) = sigmas;

end
```

Plotting

```
figure()

hold on
for i = 1:3
    e_mu = abs( mus(:, i) - S(i) ) ./ mus(:, i);
    e_sig = abs( sigmas_total(:, i) - D(i) ) ./ sigmas_total(:, i);
    subplot(3, 2,2*i -1)

    plot(test_points, e_mu);
    if i ==1
```

```
        title("Mu Errors")
    end
    ylabel("Mu_" + string(i));

    if i == 3
        xlabel("R")
    end


    subplot(3, 2,2*i)
    plot(test_points, e_sig);
    if i ==1
        title("Sigma Errors")
    end
    ylabel("Sigma_" + string(i));

    if i == 3
        xlabel("R")
    end


end
```

Fns

```
function f = myIshigami(theta)
    a = 7;
    b = 0.1;
    f = sin(theta(1)) + a * (sin(theta(2)))^2 + b*theta(3)^4 * sin(theta(1));
end
```

Published with MATLAB® R2024b

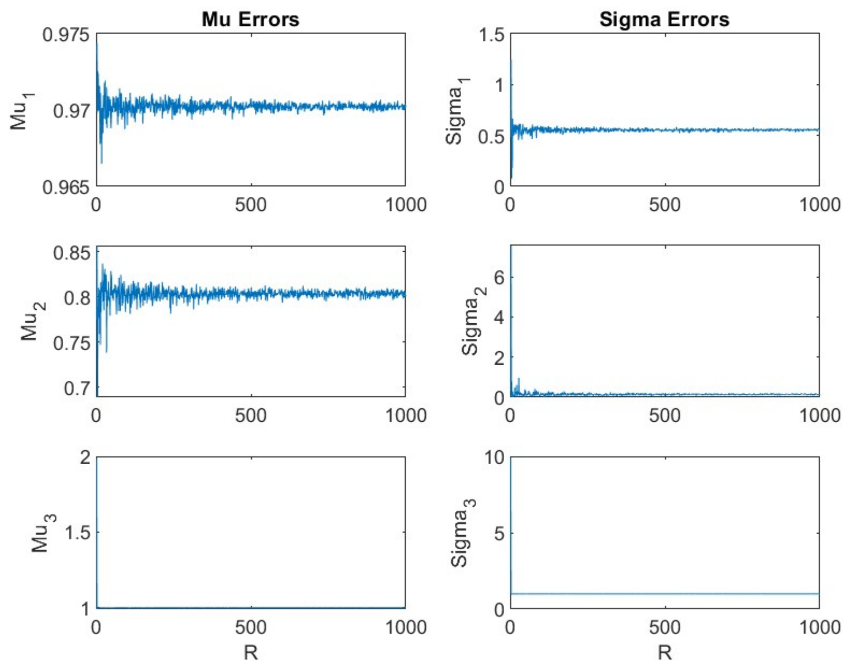


Figure 1: Q4 Plots

5 Q5

Table of Contents

.... 1

```
clc; clearvars; close all
c = [2 1];


sigma = [1, 3];
% Eqn 9.29
for i = 1:2
    D(i) = c(i)^2 * sigma(i)^2;
end

% Eqn 9.18
S = D/sum(D)

D
```

```
S =

    0.3077    0.6923


D =

     4     9
```

Published with MATLAB® R2024b